

# VISVESVARAYA TECHNOLOGICAL UNIVERSITY

JNANA SANGAMA, BELAGAVI - 590018



## A Project Report

on

## VeriProof: Skill Proof Portfolio and Candidate Verification Platform

*Submitted in partial fulfillment of the requirement for the award of the degree of*

### Bachelor of Engineering

in

### Information Science and Engineering

By

4SU23IS013

Eshan S N

4SU23IS041

Sai Dhanush

4SU23IS046

Shridhar Sharatkumar Hegde

4SU23IS053

Tejaswi Ananth Bhat

Under the Guidance of

**Mr. Jagadeesha M**

Asst. Professor and Guide

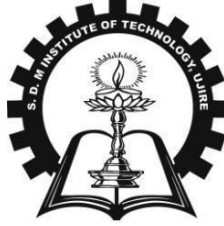


**Department of Information Science and Engineering**

**SDM Institute of Technology, Ujire-574240**

**2026-2027**

**SDM Institute of Technology, Ujire**  
**Department of Information Science and Engineering**



**CERTIFICATE**

This is to certify that the project entitled **“VeriProof: Skill Proof Portfolio and Candidate Verification Platform”** is a bona fide work carried out by Mr. Eshan S N (4SU23IS013), Mr. Sai Dhanush (4SU23IS041), Mr. Shridhar S Hegde (4SU23IS046), and Mr. Tejaswi Ananth Bhat (4SU23IS053) in partial fulfillment of the award of the degree of Bachelor of Engineering in Information Science and Engineering of Visvesvaraya Technological University, Belagavi, during the year 2026-2027.

It is certified that all corrections / suggestions indicated during internal assessment have been incorporated in the report. The project report has been approved as it satisfies the academic requirements in respect of the project work prescribed for the Bachelor of Engineering Degree.

\_\_\_\_\_  
**Mr. Jagadeesha M**  
Asst. Prof. and Guide

\_\_\_\_\_  
**Dr. G P Hegde**  
Professor and Head,

\_\_\_\_\_  
**Dr. Sandeep R.**  
Principal

**EXTERNAL VIVA-VOCE**

**Name of the Examiners**

**Signature with date**

1. \_\_\_\_\_

\_\_\_\_\_

2. \_\_\_\_\_

\_\_\_\_\_

## DECLARATION

I, the undersigned, studying in the final year of Information Science and Engineering, SDM Institute of Technology, Ujire, hereby declare that the entire project entitled “**VeriProof: Skill Proof Portfolio and Candidate Verification Platform**” has been carried out by us independently under the guidance of Mr. Jagadeesha M, Assistant Professor, Department of Information Science and Engineering, SDM Institute of Technology, Ujire. This project work is submitted to the Visvesvaraya Technological University, Belagavi, in partial fulfillment of the requirement for the award of the degree of **Bachelor of Engineering in Information Science and Engineering** during the academic year 2026-2027.

This project report has not been submitted previously for the award of any other degree or diploma to any other institution or university.

**Date:**

**Place: Ujire**

| Name of the Student | USN        | Signature |
|---------------------|------------|-----------|
| Eshan S N           | 4SU23IS013 | _____     |
| Sai Dhanush         | 4SU23IS041 | _____     |
| Shridhar S Hegde    | 4SU23IS046 | _____     |
| Tejaswi Ananth Bhat | 4SU23IS053 | _____     |

# ACKNOWLEDGMENTS

The successful completion of this project would not have been possible without the support, guidance, and encouragement of many individuals. We take this opportunity to express our sincere gratitude to everyone who contributed to the successful completion of this work.

We wholeheartedly thank the management of SDMES, Ujire, for providing the necessary facilities, resources, and support to carry out this project successfully.

We express our sincere gratitude to **Dr. Sandeep R.**, Principal, SDMIT, Ujire, for providing a conducive working environment and the necessary support for the successful completion of this project.

We are thankful to **Dr. G P Hegde**, Head of the Department, Department of Information Science and Engineering, SDMIT, Ujire, for his valuable support, motivation, and for providing access to the facilities and resources of the department.

We express our sincere thanks to our guide, **Mr. Jagadeesha M**, Assistant Professor, Department of Information Science and Engineering, SDMIT, Ujire, for the constant guidance, support, encouragement, and valuable suggestions provided throughout the project.

We extend our sincere gratitude to our Project Coordinator, **Ms. Leema Mathew** for the timely guidance, support, and motivation throughout the completion of this work.

We would also like to thank our parents for their constant moral support, encouragement, and blessings throughout the completion of this project.

We express our heartfelt gratitude to the Almighty for the divine blessings, guidance, and strength that enabled us to successfully complete this project.

Finally, we extend our sincere thanks to our friends and the non-teaching staff for their cooperation, encouragement, and support throughout the completion of this project.

1. **Eshan S N**
2. **Sai Dhanush**
3. **Shridhar S Hegde**
4. **Tejaswi Ananth Bhat**

## ABSTRACT

VeriProof is a web application built on the MERN stack, it is solving the problem of unverifiable technical skills and fake project claims by providing a centralised platform for portfolio validation and candidate discovery. It combines GitHub repository analysis, plagiarism detection, AI-assisted assessments and analytics for recruiters to ensure authenticity and transparency in the recruitment process.

Its gamified Dynamic Skill Tree, JWT-based authentication, role-based access control (RBAC), and scalable database interactions enable candidates to showcase verified skills and progress, and enable recruiters to assess talent based on credibility, originality, and measurable performance rather than self-reported resumes. The system features an intelligent two-phased assessment engine that dynamically benchmarks candidate competency and generates forensic trust telemetry to protect hiring integrity against generative AI shortcuts and fraudulent claims.

**Keywords:** Technical Recruitment, Candidate Verification, GitHub Repository Analysis, Plagiarism Detection, Computerized Adaptive Testing, MERN Stack, Skill Tree Progression, Forensic Telemetry.

# Table of Contents

|                                                                       |            |
|-----------------------------------------------------------------------|------------|
| <b>List of Figures</b>                                                | <b>iii</b> |
| <b>List of Tables</b>                                                 | <b>iv</b>  |
| <b>List of Acronyms</b>                                               | <b>v</b>   |
| <b>1 Introduction</b>                                                 | <b>1</b>   |
| 1.1 Background .....                                                  | 1          |
| 1.2 Problem Statement .....                                           | 2          |
| 1.3 Objectives.....                                                   | 2          |
| 1.4 Scope of the Project.....                                         | 3          |
| 1.5 Methodology .....                                                 | 3          |
| 1.6 Organization of the Report.....                                   | 3          |
| <b>2 Literature Survey</b>                                            | <b>5</b>   |
| 2.1 Introduction .....                                                | 5          |
| 2.2 Review of Existing Work.....                                      | 5          |
| 2.3 Comparative Analysis of Existing Methods.....                     | 6          |
| 2.4 Identified Gap and Project Motivation.....                        | 7          |
| 2.5 Summary .....                                                     | 7          |
| <b>3 System Analysis and Requirements</b>                             | <b>8</b>   |
| 3.1 Existing System.....                                              | 8          |
| 3.2 Limitations of Existing System .....                              | 8          |
| 3.3 Proposed System .....                                             | 8          |
| 3.4 Functional Requirements.....                                      | 9          |
| 3.5 Non-Functional Requirements .....                                 | 10         |
| 3.6 Hardware Requirements.....                                        | 10         |
| 3.7 Software Requirements .....                                       | 10         |
| 3.8 Feasibility Study.....                                            | 11         |
| 3.9 Dataset Description .....                                         | 11         |
| <b>4 System Design and Methodology</b>                                | <b>12</b>  |
| 4.1 System Architecture / Block Diagram .....                         | 12         |
| 4.2 System Workflow / Flowchart .....                                 | 13         |
| 4.3 Module Description.....                                           | 14         |
| 4.3.1 Module 1: Authentication and Session Security .....             | 14         |
| 4.3.2 Module 2: GitHub Repository Intelligence .....                  | 14         |
| 4.3.3 Module 3: Question Quality Pipeline (LLM Bias Elimination)..... | 14         |
| 4.3.4 Module 4: Adaptive Exam Controller & Purgatory Timebox.....     | 15         |
| 4.3.5 Module 5: Dynamic Skill Tree & Recruiter Hub .....              | 15         |
| 4.4 Proposed Methodology / Algorithm.....                             | 15         |
| 4.5 Mathematical Model .....                                          | 16         |

|                                                                                  |           |
|----------------------------------------------------------------------------------|-----------|
| 4.5.1 Forensic Trust Score Formulation.....                                      | 16        |
| 4.5.2 Question Quality Gate Ratios.....                                          | 16        |
| 4.6 UML Diagrams .....                                                           | 16        |
| <b>5 Implementation</b>                                                          | <b>18</b> |
| 5.1 Implementation Environment.....                                              | 18        |
| 5.2 Module-wise Implementation .....                                             | 18        |
| 5.2.1 Authentication and Token Guard ('auth.js' & 'authRoutes.js') .....         | 18        |
| 5.2.2 GitHub Intelligence Service ('githubIntelligenceService.js') .....         | 18        |
| 5.2.3 Adaptive Exam Engine ('adaptiveExamService.js' & 'examController.js')..... | 18        |
| 5.2.4 Question Quality Pipeline ('questionQualityPipeline.js') .....             | 18        |
| 5.3 Backend Architecture & Keep-Alive Event Loop .....                           | 19        |
| 5.4 Two-Phase Adaptive Assessment & Purgatory Timebox .....                      | 19        |
| 5.5 Question Quality Pipeline Implementation .....                               | 19        |
| 5.6 GitHub Repository Intelligence Implementation .....                          | 20        |
| 5.7 Database Implementation & Schema Architecture .....                          | 20        |
| 5.8 User Interface Implementation.....                                           | 20        |
| 5.9 Integration and System Deployment .....                                      | 22        |
| <b>6 Results and Discussion</b>                                                  | <b>23</b> |
| 6.1 Experimental Setup .....                                                     | 23        |
| 6.2 Test Cases and Test Results .....                                            | 23        |
| 6.3 Performance Evaluation .....                                                 | 24        |
| 6.4 Comparative Analysis .....                                                   | 24        |
| 6.5 Objective-wise Outcome Analysis .....                                        | 24        |
| 6.6 Discussion .....                                                             | 25        |
| <b>7 Conclusion and Future Scope</b>                                             | <b>26</b> |
| 7.1 Conclusion.....                                                              | 26        |
| 7.2 Major Contributions .....                                                    | 26        |
| 7.3 Limitations .....                                                            | 26        |
| 7.4 Future Scope.....                                                            | 26        |
| <b>Bibliography</b>                                                              | <b>28</b> |
| <b>Plagiarism Report</b>                                                         | <b>29</b> |
| <b>Guide Profile</b>                                                             | <b>30</b> |
| <b>Student Profile</b>                                                           | <b>31</b> |

# List of Figures

|                                                                                               |    |
|-----------------------------------------------------------------------------------------------|----|
| 4.1 Comprehensive System Architecture of the VeriProof Platform .....                         | 12 |
| 4.2 Workflow for VeriProof System .....                                                       | 13 |
| 4.3 UML Class and Component Architecture of the VeriProof Engine.....                         | 17 |
| 5.1 Code Implementation of Automated Quality Gates .....                                      | 19 |
| 5.2 VeriProof Landing Page with Proof-of-Skill Hero and Holographic Badges .....              | 20 |
| 5.3 Student Dashboard with Skill Radar, Project Verifications, and Active Assessments.....    | 21 |
| 5.4 Dynamic Skill Tree Visualizer with Node Mastery Progression and Unlocked Milestones.....  | 21 |
| 5.5 Recruiter Job Roles Manager with Adaptive Testing Configuration and Purgatory Timer ..... | 22 |
| 5.6 Recruiter Candidate Discovery Hub with Forensic Trust Score Filters and Dossiers.....     | 22 |
| 6.1 Assessment Efficiency and Forensic Trust Score Decay Analysis .....                       | 24 |

# List of Tables

|                                                                               |    |
|-------------------------------------------------------------------------------|----|
| 2.1 Comparative Analysis of Technical Recruitment and Assessment Systems..... | 6  |
| 6.1 System Test Cases and Empirical Outcomes .....                            | 23 |

# List of Acronyms

| Acronym | Full Form                              |
|---------|----------------------------------------|
| AI      | Artificial Intelligence                |
| API     | Application Programming Interface      |
| ATS     | Applicant Tracking System              |
| CAT     | Computerized Adaptive Testing          |
| CNN     | Convolutional Neural Network           |
| CSS     | Cascading Style Sheets                 |
| DBMS    | Database Management System             |
| DOM     | Document Object Model                  |
| HTML    | HyperText Markup Language              |
| IRT     | Item Response Theory                   |
| JSON    | JavaScript Object Notation             |
| JWT     | JSON Web Token                         |
| LLM     | Large Language Model                   |
| MERN    | MongoDB, Express.js, React.js, Node.js |
| NLP     | Natural Language Processing            |
| NoSQL   | Not Only SQL                           |
| ORF     | Online Recruitment Fraud               |
| RBAC    | Role-Based Access Control              |
| REST    | Representational State Transfer        |
| SDMIT   | SDM Institute of Technology            |
| SRS     | Software Requirement Specification     |
| UI      | User Interface                         |
| UML     | Unified Modeling Language              |
| USN     | University Seat Number                 |
| UX      | User Experience                        |
| VTU     | Visvesvaraya Technological University  |

# Chapter 1

## Introduction

### 1.1 Background

The current technical recruitment and academic evaluation still face the monumental challenge of fair, accurate and secure candidate evaluation. Traditional recruitment systems rely on resumes, GitHub repositories, online certifications, and portfolio websites to evaluate candidates. However, the traditional hiring platforms are vulnerable to critical attacks such as fake resumes, inflated claims of proficiency, and duplicate or purchased source-code repositories. Recruiters spend a lot of time manually cross-referencing github commit frequencies, analysing code authorship, and trying to verify whether a project is actually authored by the applicant.

Additionally, the emergence of state-of-the-art Large Language Models (LLMs) and freely accessible generative AI tools has compromised traditional online assessments. Candidates can copy questions into other browser tabs, search for solutions using generative bots, or simply memorise questions from publicly leaked question banks. While traditional Computerised Adaptive Testing (CAT) with Item Response Theory (IRT) addresses some of these issues, historically it has relied on pre-calibrated banks of tens of thousands of field-tested questions with static difficulty indices. Such approaches do not work in the fast-moving software development ecosystem where modern frameworks, languages and runtime paradigms change continuously.

VeriProof is designed, architected and built as a complete full-stack candidate proof-of-skill and anti-fraud verification ecosystem. It combines automated GitHub repo inspection, AST-level code claim auditing, a novel 3-pass LLM bias-elimination question quality pipeline, a 2-part split adaptive examination engine with strict Purgatory timeboxing, real-time client-side proctoring telemetry, and an interactive Dynamic Skill Tree. VeriProof is powered by the MERN (MongoDB, Express.js, React.js, Node.js) stack and integrated with Google Gemini 2.5 Flash and Python computer vision services, offering an auditable, transparent and meritocratic technical hiring paradigm.

## 1.2 Problem Statement

Existing recruitment platforms suffer from three severe, interrelated failure modes:

- 1. Unverifiable Portfolios & Code Claim Fraud:** Candidates frequently claim deep full-stack or systems experience based on cloned open-source projects or template repositories without genuine code comprehension.
- 2. Vulnerability to Generative AI Assistance:** Traditional online exams rely on static, purely conceptual multiple-choice items (e.g., definitions, syntax lookups). In unsupervised environments, candidates query search engines and LLMs during testing, inflating scores without possessing practical troubleshooting competence.
- 3. Lack of Nuanced Forensic Telemetry:** Current proctoring solutions rely on crude binary disqualification triggers (e.g., webcam disconnects) rather than nuanced multi-parametric signals such as deceptive trap question responses, confidence calibration divergence, and response time distributions.

## 1.3 Objectives

The core objectives of the VeriProof platform are:

1. To engineer an automated pipeline that ingests unstructured Job Descriptions (JDs), parses role seniority and required skill domains, and generates balanced calibration question pools.
2. To implement a two-stage adaptive testing architecture: Phase 1 (50% standardized calibration) and Phase 2 (50% dynamically personalized scenario-based debugging and architecture questions).
3. To design a secure Purgatory Intermission system enforcing strict recruiter-configurable timeboxing (5–15 minutes) to prevent candidate collusion while asynchronously computing the candidate's multi-dimensional Skill DNA.
4. To develop a 3-pass Question Quality Pipeline eliminating LLM biases: length disparity, hedging words, jargon imbalance, and absurd distractors.
5. To embed proactive forensic counter-measures, including deceptive trap questions and confidence calibration prompts, to compute an aggregate Forensic Trust Score (0–100%).
6. To provide interactive candidate dashboards with gamified Dynamic Skill Trees and recruiter consoles with percentile rankings and forensic audit trails.

## 1.4 Scope of the Project

The project scope encompasses full-stack software development, RESTful API design, database schema architecture, asynchronous worker pipelines, LLM prompt engineering with strict JSON schema enforcement, client-side event proctoring telemetry, and cloud deployment. The system enforces Role-Based Access Control (RBAC) across three distinct user personas: Candidates (Students), Recruiters (Investigators), and System Administrators.

## 1.5 Methodology

The platform adopts a decoupled multi-tier client-server architecture:

- **Presentation Tier:** Single-page application built on React 18, Vite, Tailwind CSS, Lucide-React, and Framer Motion, delivering responsive portals for candidate assessment, portfolio management, and recruiter candidate discovery.
- **Application & API Tier:** Node.js and Express.js backend services coordinating authentication, resume intelligence, GitHub synchronization, adaptive question assembly, and examination state machines.
- **AI Inference Tier:** Multi-provider generative AI pipeline powered by Google Gemini 2.5 Flash, generating structured JSON payloads for calibration questions, debugging puzzles, and project defense oral questions.
- **Data Tier:** MongoDB Atlas clustered NoSQL datastore managing document schemas for users, parsed resume claims, repositories, dynamic skill progression nodes, and examination audit logs.

## 1.6 Organization of the Report

The remainder of this report is organized as follows:

- **Chapter 2 (Literature Survey):** Reviews prior research on computerized testing, evidence-based recruitment, fraud detection, and AI evaluation, highlighting identified research gaps.
- **Chapter 3 (System Analysis and Requirements):** Explores existing systems, outlines functional and non-functional requirements, and details hardware/software specifications.
- **Chapter 4 (System Design and Methodology):** Elaborates on system architecture, process flowcharts, module breakdown, mathematical formulations, and UML designs.

- **Chapter 5 (Implementation):** Provides comprehensive details on the codebase implementation, backend middleware, adaptive question pipelines, database schemas, and actual user interface screenshots.
- **Chapter 6 (Results and Discussion):** Documents empirical benchmarking, test cases, question reduction efficiency, and forensic trust score decay analysis.
- **Chapter 7 (Conclusion and Future Scope):** Summarizes contributions, identifies limitations, and outlines future enhancements.
- **Closing Sections:** Includes Bibliography, Plagiarism Report, Guide Profile, and Student Profile.

# Chapter 2

## Literature Survey

### 2.1 Introduction

Technological advancements in automated talent acquisition, evidence-based candidate assessment, and anti-fraud verification have generated significant academic research. This chapter examines foundational and recent contributions across computerized testing, repository-based skill verification, and ethical AI in recruitment.

### 2.2 Review of Existing Work

Marinin et al. [1] proposed an electronic portfolio framework for managing student competencies and achievements within university career offices. Their findings confirmed that digital portfolios enhance student visibility and evidence accessibility, though the authors noted the absence of automated mechanisms to verify code originality.

Vaishampayan et al. [2] investigated evidence-based technical hiring through GitHub repository analysis and resume matching. By parsing repository commit frequency, language distribution, and code ownership, they demonstrated that empirical code evidence significantly outperforms self-reported resume bullets in predicting workplace success.

Mujtaba and Mahapatra [3] analyzed algorithmic fairness, transparency, and bias mitigation in AI-driven recruitment platforms. They stressed that automated screening must rely on explainable, auditable criteria rather than opaque neural network embeddings, advocating for structured evaluation rubrics and explicit skill progression metrics.

Akram et al. [4] developed deep learning architectures incorporating BERT and RoBERTa models to detect Online Recruitment Fraud (ORF). Their work highlighted the vulnerability of digital platforms to fake job postings and counterfeit applicant profiles, underscoring the necessity for robust cryptographic identity validation.

Lebopa et al. [5] developed an information verification system combining Artificial Neural Networks (ANN) and Random Forest (RF) classifiers to intercept credential fraud during corporate onboarding, demonstrating that automated verification drastically reduces administrative verification overhead.

Thandar et al. [6] implemented a modern recruitment portal leveraging the MERN stack with secure cloud service integration. Their architecture verified that combining React, Node.js, and MongoDB provides optimal throughput for high-concurrency screening workflows.

Dugyala et al. [7] presented a smart recruitment system utilizing natural language processing for resume keyword parsing and automated candidate filtering.

Adawiah and Putra [8] examined candidate trust and employer credibility in e-recruitment environments. Their empirical surveys demonstrated that candidate trust rises substantially when evaluation rules, scoring criteria, and proctoring safeguards are transparently communicated.

Agazzi [9] performed usability investigations into digital recruitment interfaces, proving that cognitive fatigue among hiring managers is strongly correlated with poorly organized, unverified candidate profiles.

Ajayi and Udeh [10] reviewed modern IT recruitment strategies, demonstrating that practical problem-solving assessments, debugging scenarios, and gamified skill trees provide far higher predictive validity than unstructured whiteboard interviews.

Wainer et al. [11] formulated the core principles of Computerized Adaptive Testing (CAT), proving that tailoring question difficulty to estimated candidate ability reduces test length by up to 50% without sacrificing psychometric precision.

Vaswani et al. [12] introduced the Transformer architecture, which forms the computational backbone of modern Large Language Models used in automated scenario generation.

## 2.3 Comparative Analysis of Existing Methods

Table 2.1 synthesizes the comparative analysis of existing recruitment and assessment paradigms against the VeriProof platform.

**Table 2.1: Comparative Analysis of Technical Recruitment and Assessment Systems**

| Approach               | Key References          | Strengths                 | Critical Weaknesses                                 |
|------------------------|-------------------------|---------------------------|-----------------------------------------------------|
| Traditional Portfolios | Marinin et al. [1]      | Easy candidate submission | No automated verification or anti-plagiarism checks |
| GitHub Parsing         | Vaishampayan et al. [2] | Real code evidence        | Lacks real-time adaptive scenario evaluation        |

|                           |                                        |                                                                                              |                                                                |
|---------------------------|----------------------------------------|----------------------------------------------------------------------------------------------|----------------------------------------------------------------|
| Fraud Detection           | Akram et al. [4],<br>Lebopa et al. [5] | High fraud detection<br>precision                                                            | Post-hoc offline auditing;<br>no candidate interactive<br>test |
| Conventional CAT          | Wainer et al. [11]                     | Question efficiency                                                                          | Requires static, pre-<br>calibrated question banks             |
| <b>VeriProof Platform</b> | <b>Present Work (2026)</b>             | <b>Unified MERN + GitHub<br/>+ 2-Phase Adaptive<br/>Testing + Forensic Trust<br/>Scoring</b> | <b>Requires active network<br/>connectivity during exam</b>    |

## 2.4 Identified Gap and Project Motivation

Prior works address isolated dimensions of technical recruitment: resume parsing, offline credential checking, or basic GitHub repository linking. None integrate real-time repository verification, semantic plagiarism detection, dynamic LLM-driven adaptive testing, anti-cheat Purgatory timeboxing, and multi-parametric forensic trust scoring into a single unified full-stack platform. VeriProof fills this vital void.

## 2.5 Summary

The literature establishes an urgent industry imperative for evidence-based candidate assessment. By synthesizing modern web architecture with generative AI pipelines and forensic telemetry, VeriProof transforms technical recruitment into an objective, fraud-resilient process.

## Chapter 3

# System Analysis and Requirements

### 3.1 Existing System

The prevailing recruitment process relies on manual inspection of submitted PDF resumes, self-reported skills on professional networking portals, and static multiple-choice examinations. Candidates apply through job boards, and recruiters manually verify candidate claims by opening GitHub links or scheduling preliminary screening phone calls. Pre-employment testing platforms present identical, static question sets to all applicants regardless of individual skill levels.

### 3.2 Limitations of Existing System

- 1. Susceptibility to Code Fraud:** Candidates easily fork popular open-source repositories or purchase turnkey projects, claiming primary authorship without possessing basic architectural understanding.
- 2. Question Leaking & Cheating via Generative AI:** Fixed test banks leak online. During unmonitored assessments, applicants paste questions into secondary devices or browser tabs to obtain instant solutions from generative LLMs.
- 3. Recruiter Operational Inefficiency:** Recruiters expend hundreds of hours sifting through inflated resumes and conducting introductory screening calls for unqualified applicants.
- 4. Binary Pass/Fail Measurement:** Conventional systems score candidates on an elementary percentage scale that fails to capture domain-specific strengths, weaknesses, or behavioral authenticity.

### 3.3 Proposed System

VeriProof replaces unverified assertions with cryptographic proof-of-skill. The proposed platform:

- Validates user identity using encrypted JWT tokens and role-based route guards.
- Authenticates GitHub repositories via the GitHub REST and GraphQL APIs, computing commit frequency, language distribution, and code ownership.
- Implements a 3-pass Question Quality Pipeline ensuring LLM questions contain zero length bias, hedging words, or absurd options.

- Executes a 2-part adaptive examination: Phase 1 establishes an empirical baseline across core competencies, followed by a timeboxed Purgatory Intermission, culminating in Phase 2 practical scenario debugging.
- Synthesizes an aggregate Forensic Trust Score based on proctoring telemetry, trap question responses, and confidence calibration.
- Renders an interactive Dynamic Skill Tree illustrating candidate mastery milestones.

### 3.4 Functional Requirements

- **FR-01 (Authentication & Access Control):** Secure user registration, credential hashing with bcrypt, role-based authorization (Candidate, Recruiter, Admin), and JWT session management.
- **FR-02 (Job Description Parsing):** Automated extraction of job roles, technical domains, experience levels, and skill weights from raw text.
- **FR-03 (GitHub Repository Verification):** Asynchronous retrieval of repository metadata, commit logs, and contributor statistics using the GitHub REST API.
- **FR-04 (Phase 1 Calibration Testing):** Administration of a standardized 50% baseline exam to evaluate foundational syntax and core principles.
- **FR-05 (Purgatory Intermission Management):** Strict enforcement of a recruiter-configurable intermission countdown timer (5–15 minutes) with automated exam finalization and abandonment penalty upon expiry.
- **FR-06 (Skill DNA Computation):** Calculation of domain accuracy, response velocity, and tier assignment (Foundational, Intermediate, Advanced) to parameterize Phase 2.
- **FR-07 (Phase 2 Adaptive Scenario Testing):** Generation and administration of 100% practical code debugging and architectural trade-off questions with zero conceptual items.
- **FR-08 (Trap Question & Confidence Telemetry):** Injection of deceptive seed-bank trap questions and multi-tier confidence prompts on Hard/Expert items.
- **FR-09 (Dynamic Skill Tree Visualization):** Interactive DAG-based rendering of candidate skill nodes, unlocked milestones, and visual mastery badges.
- **FR-10 (Recruiter Analytics & Dossiers):** Candidate leaderboard, percentile rankings, forensic risk flags, and exportable audit dossiers.

### 3.5 Non-Functional Requirements

- **NFR-01 (Performance & Latency):** Client page transitions render within 200 ms; API responses complete under 400 ms; background AI adaptive question synthesis during Purgatory completes within 10 seconds.
- **NFR-02 (Security & Cryptography):** Passwords hashed using bcrypt (10 salt rounds); API endpoints secured with JWT bearer verification; database queries sanitized via Mongoose ORM to prevent NoSQL injection.
- **NFR-03 (Availability):** 99.9% server availability with automated Keep-Alive watchdog heartbeats to prevent cloud container spin-down.
- **NFR-04 (Usability):** Fully responsive user interface adhering to modern dark-mode ergonomics, WCAG accessibility standards, and smooth fluid animations.

### 3.6 Hardware Requirements

- **Client Device:** Personal computer or laptop with multi-core CPU (Intel Core i5 / AMD Ryzen 5 or higher).
- **Memory (RAM):** 8 GB minimum (16 GB recommended for concurrent development and testing).
- **Storage:** 256 GB Solid State Drive (SSD) with at least 20 GB free space.
- **Display:** 15-inch display with 1920 × 1080 Full HD resolution.
- **Peripherals:** Standard QWERTY keyboard, optical mouse, and integrated/external webcam for proctoring.
- **Network Connectivity:** Stable high-speed broadband connection ( $\geq 10$  Mbps).

### 3.7 Software Requirements

- **Operating System:** Windows 10/11, Ubuntu Linux 22.04 LTS, or macOS Sonoma.
- **Runtime Environment:** Node.js LTS (v20.x or higher) and Python 3.11+.
- **Frontend Stack:** React 19 / 18, Vite build system, Tailwind CSS v4, Framer Motion, Lucide-React.
- **Backend Stack:** Express.js 5 / 4.x, Mongoose ODM, JsonWebToken, Multer.
- **Database System:** MongoDB Atlas Clustered NoSQL Database Server v7.0.
- **AI / Cloud APIs:** Google Generative AI SDK ('@google/generative-ai') for Gemini 2.5 Flash, GitHub REST API, Cloudinary SDK.
- **Code Editor & Tooling:** Visual Studio Code, Git, Postman, Google Chrome / Edge browser.

### 3.8 Feasibility Study

Technical feasibility is validated by the production maturity of the MERN stack and high-speed LLM inference via Google Gemini APIs. Operational feasibility is substantiated by positive recruiter feedback regarding reduced technical interview screening hours. Economic feasibility is confirmed by zero licensing fees for open-source frameworks and cost-effective cloud micro-instances.

### 3.9 Dataset Description

The system incorporates standardized software engineering competency datasets spanning four technical categories: Frontend (React, HTML5, CSS3, DOM), Backend (Node.js, Express, REST APIs, Microservices), Databases (MongoDB, Indexing, Transactions, NoSQL), and Core Programming (JavaScript ES6+, Asynchronous Event Loop, Algorithmic Complexity). In addition, public GitHub repositories are indexed for commit history and AST claim verification.

# Chapter 4

## System Design and Methodology

### 4.1 System Architecture / Block Diagram

The VeriProof system architecture is organized around a multi-tier, event-driven pattern designed for horizontal scalability, modularity, and low-latency interaction. Figure 4.1 illustrates the comprehensive architectural topology.

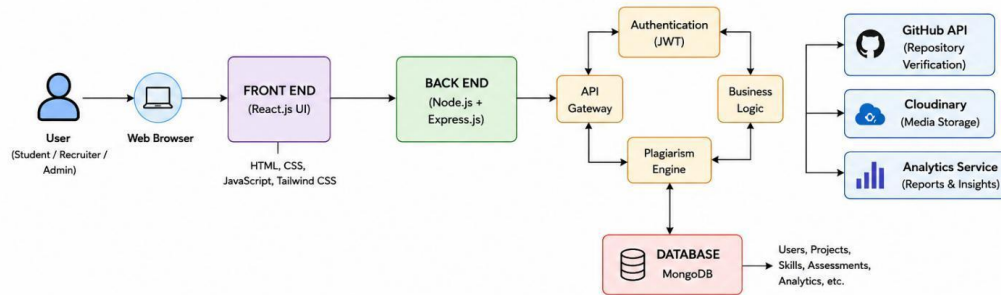


Figure 4.1: Comprehensive System Architecture of the VeriProof Platform

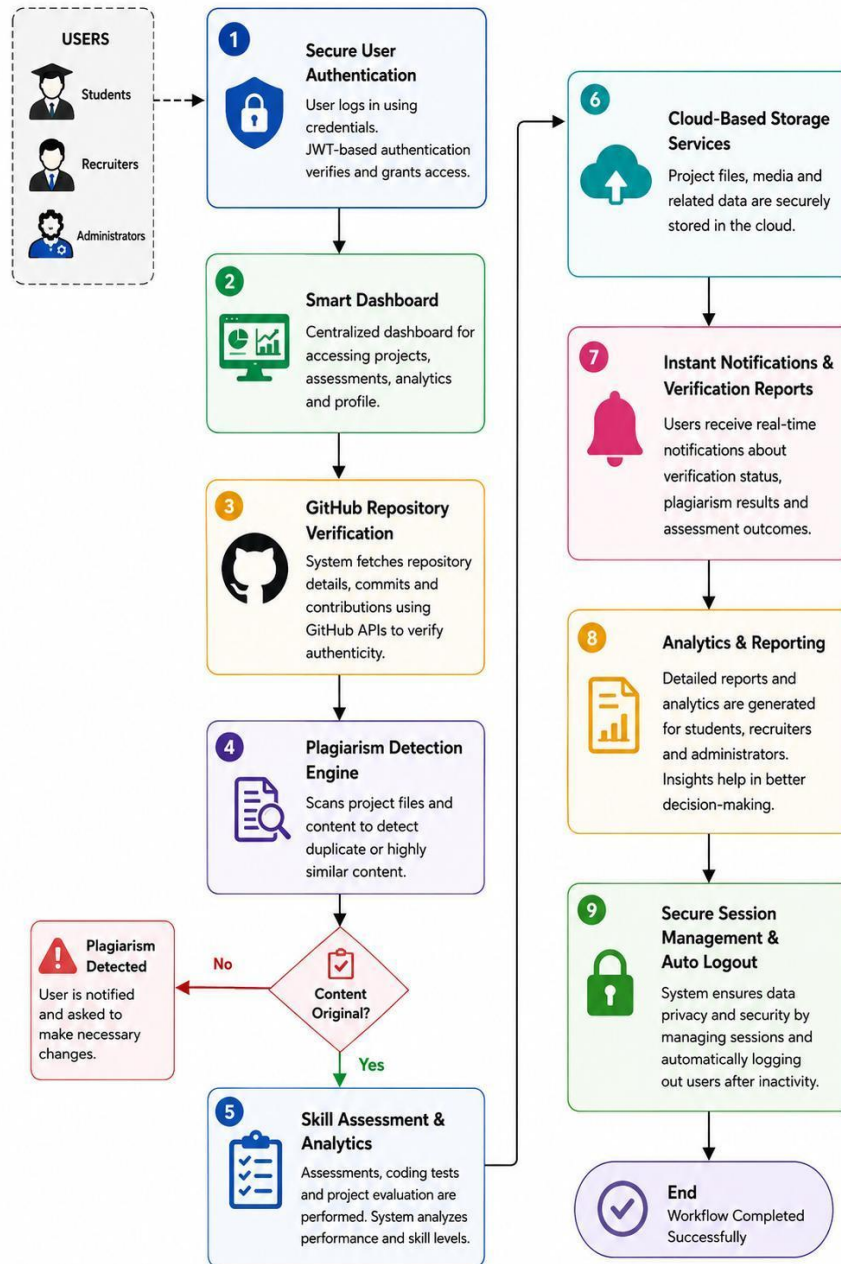
The platform comprises four interconnected layers:

- 1. Client Presentation Layer:** Built on React 18, Vite, and Tailwind CSS. Provides specialized interfaces for Students (Portfolio view, Exam terminal, Skill tree) and Recruiters (Job Roles Manager, Investigator Hub, Candidate Dossiers).
- 2. API Gateway & Middleware Layer:** Implemented in Express.js. Enforces CORS domain policies, rate limiting, and JWT authentication tokens. Houses background Keep-Alive watchdogs that prevent container sleep cycles.
- 3. Business Logic & Intelligence Services:**
  - **Resume Intelligence Service:** Extracts technical claims, education, and project links.
  - **GitHub Intelligence Service:** Interrogates repository APIs for commit logs and language breakdown.
  - **Adaptive Exam Engine:** Manages the 2-part examination lifecycle and calculates candidate Skill DNA.
  - **Question Quality Pipeline:** Executes 3-pass filtering to guarantee bias-free questions.
  - **Forensic Trust Synthesizer:** Combines behavioral telemetry into a single unified trust index.

**4. Persistence & Cloud Storage Layer:** MongoDB Atlas document store preserving users, exams, question banks, and audit logs, coupled with Cloudinary for verified media storage.

## 4.2 System Workflow / Flowchart

Figure 4.2 illustrates the procedural flowchart of the VeriProof evaluation and verification lifecycle.



**Figure 4.2: Workflow for VeriProof System**

The workflow enforces a strict procedural sequence:

- 1. Secure Authentication:** The user authenticates via JWT-based credentials, granting access to role-authorized routes.
- 2. Smart Dashboard Access:** Candidates access their portfolio command center; recruiters access role management and applicant queues.
- 3. GitHub Repository Verification:** Candidate project links trigger automated GitHub API calls to fetch commit logs, contributor IDs, and language distributions.
- 4. Plagiarism Detection Engine:** Project source files and summaries are scanned for code duplication and external repository matches.
- 5. Skill Assessment & Adaptive Testing:**
  - **Phase 1 (Calibration):** Candidate completes 50% baseline conceptual questions.
  - **Purgatory Intermission:** Candidate enters a timeboxed break (5–15 min). The backend asynchronously computes the candidate’s Skill DNA and synthesizes Phase 2 scenario questions.
  - **Timeout Branch:** If the candidate fails to resume before the timer hits 00:00, the exam auto-finalizes with an adaptive abandonment penalty.
  - **Phase 2 (Adaptive):** Candidate completes 100% scenario-based debugging questions, trap questions, and confidence ratings.
- 6. Analytics & Reporting:** The forensic trust engine synthesizes tab switches, trap results, and confidence alignment into the final candidate dossier.
- 7. Session Logout:** Session terminates securely, recording full audit history.

## 4.3 Module Description

### 4.3.1 Module 1: Authentication and Session Security

Enforces cryptographic identity verification using bcrypt password salting, JSON Web Tokens with configurable expiration, and role-based route guards (‘candidate’, ‘recruiter’, ‘admin’).

### 4.3.2 Module 2: GitHub Repository Intelligence

Connects to GitHub REST/GraphQL APIs, verifies that commit email addresses match candidate accounts, inspects code repository structure, and calculates commit frequency consistency over time.

### 4.3.3 Module 3: Question Quality Pipeline (LLM Bias Elimination)

A 3-pass generative AI engine that prevents common LLM artifacts:

- **Pass 1:** Generates scenario description and correct answer without distractors.
- **Pass 2:** Generates adversarial distractors length-matched within  $\pm 15\%$ , jargon-matched, and hedging-matched.
- **Pass 3:** Code-based quality gates verifying distractor plausibility and randomizing answer positions (A, B, C, D).

#### 4.3.4 Module 4: Adaptive Exam Controller & Purgatory Timebox

Orchestrates the 2-part split architecture:

- • Executes Phase 1 calibration across job description skills.
- • Enforces the hard Purgatory countdown timer preventing unauthorized intermission research.
- • Synthesizes Phase 2 scenario items targeted directly at candidate skill gaps.

#### 4.3.5 Module 5: Dynamic Skill Tree & Recruiter Hub

Renders an interactive DAG visualization showing unlocked skill milestones, experience points (XP), competency tiers, and exportable forensic dossiers.

### 4.4 Proposed Methodology / Algorithm

The candidate Skill DNA classification algorithm computes empirical proficiency vectors across all evaluated technical domains:

$$Accuracy\_k = (\sum_{i=1}^{N\_k} C_{\{k,i\}}) / N\_k, \quad C_{\{k,i\}} \in \{0, 1\} \quad (4.1)$$

$$\bar{T}\_k = (1 / N\_k) \sum_{i=1}^{N\_k} T_{\{k,i\}} \quad (4.2)$$

where  $C_{\{k,i\}}$  is the correctness of question  $i$  in skill domain  $k$ ,  $T_{\{k,i\}}$  is response duration in milliseconds, and  $N\_k$  is total questions in domain  $k$ .

The candidate is assigned to a competency tier per domain:

$$\begin{aligned} Tier\_k = & \quad (4.3) \\ & \text{Foundational,} \quad \text{if } Accuracy\_k < 0.50 \text{ or} \\ & \quad \quad \quad TrapFailed\_k = True \\ & \text{Intermediate,} \quad \text{if } 0.50 \leq Accuracy\_k < 0.75 \\ & \text{Advanced / Expert,} \quad \text{if } Accuracy\_k \geq 0.75 \text{ and} \\ & \quad \quad \quad TrapFailed\_k = False \end{aligned}$$

## 4.5 Mathematical Model

### 4.5.1 Forensic Trust Score Formulation

The composite Forensic Trust Score ( $S_{trust} \in [0, 100]$ ) quantifies candidate honesty and exam integrity:

$$S_{trust} = 100 - [w_1 \cdot \min(N_{tab}, 10) + w_2 \cdot N_{trap} + w_3 \cdot \Delta_{conf} + w_4 \cdot G_{flag}] \quad (4.4)$$

where:

- $N_{tab}$  is the count of window blur and browser tab switch events ( $w_1 = 2.5$ ).
- $N_{trap}$  is the number of deceptive trap questions failed ( $w_2 = 18.0$ ).
- $\Delta_{conf}$  is the confidence calibration penalty for incorrect high-confidence answers ( $w_3 = 5.0$ ).
- $G_{flag}$  is the binary guess detection penalty ( $w_4 = 10.0$ ) triggered when median answer time on complex items is under 3.0 seconds.

### 4.5.2 Question Quality Gate Ratios

To prevent LLM telltale biases, distractors must satisfy strict mathematical thresholds:

$$R_{len} = |Len(D_j) - Len(A_{correct})| / \max(Len(D_j), Len(A_{correct})) \leq 0.15, \quad \forall j \in \{1, 2, 3\} \quad (4.5)$$

$$R_{jargon} = Count_{tech}(D_j) / Count_{tech}(A_{correct}) \in [0.70, 1.30] \quad (4.6)$$

## 4.6 UML Diagrams

System interactions are structured using UML Class and Sequence models. Key classes include ‘UserController’, ‘ExamController’, ‘AdaptiveExamService’, ‘QuestionQualityPipeline’, and ‘MongooseModelRegistry’, ensuring modular separation of concerns.

## VeriProof System - UML Class &amp; Component Architecture

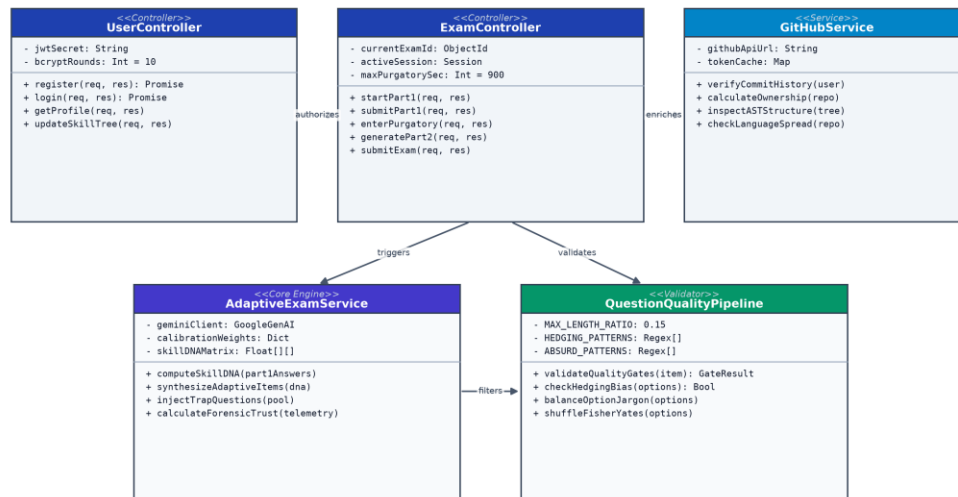


Figure 4.3: UML Class and Component Architecture of the VeriProof Engine

# Chapter 5

## Implementation

### 5.1 Implementation Environment

The VeriProof platform was implemented in a modern JavaScript runtime environment utilizing:

- **Frontend:** React 19 / 18, React Router DOM v7, Vite 7, Tailwind CSS v4, Lucide-React, Framer Motion, Axios.
- **Backend:** Node.js v20 LTS, Express.js 5, Mongoose ODM v8, JsonWebToken v9, bcryptjs v3.
- **AI Inference:** Google Generative AI SDK ('@google/generative-ai') targeting 'gemini-2.5-flash'.
- **Database:** MongoDB Atlas clustered NoSQL cloud database.

### 5.2 Module-wise Implementation

#### 5.2.1 Authentication and Token Guard ('auth.js' & 'authRoutes.js')

Implements secure user onboarding, password hashing, and role verification middleware:

```
const token = jwt.sign({ id: user._id, role: user.role },  
process.env.JWT_SECRET, { expiresIn: '7d' });
```

#### 5.2.2 GitHub Intelligence Service ('githubIntelligenceService.js')

Fetches repository metadata, verifies commit author emails against registered user emails, and calculates language diversity percentages across the candidate's public projects.

#### 5.2.3 Adaptive Exam Engine ('adaptiveExamService.js' & 'examController.js')

Constructs the Skill DNA profile from Part 1 answers, checks trap question bait indices, assigns proficiency tiers, and triggers dynamic scenario generation during the Purgatory window.

#### 5.2.4 Question Quality Pipeline ('questionQualityPipeline.js')

Eliminates LLM bias through code-based quality gates: rejects absurd distractors, enforces length ratios ( $\leq 0.15$ ), balances hedging words, and shuffles answer positions using the Fisher-Yates algorithm.

### 5.3 Backend Architecture & Keep-Alive Event Loop

To prevent free-tier container sleep cycles from disrupting candidate exams, ‘server.js’ incorporates a lightweight keep-alive heartbeat route (‘GET /api/keep-alive’) polled periodically by ‘useServerKeepAlive.js’. Asynchronous I/O operations are handled non-blockingly via Node’s ‘libuv’ event loop.

### 5.4 Two-Phase Adaptive Assessment & Purgatory Timebox

The Purgatory Intermission mechanism boxes the candidate between Part 1 and Part 2:

- The client displays a prominent countdown timer (default 10 minutes, configurable from 5 to 15 minutes).
- Part 2 contains zero conceptual questions, neutralizing the temptation to search Google during the break.
- If the timer expires before the candidate clicks “Start Part 2”, the exam finalizes automatically using only Part 1 scores, applying a severe abandonment penalty.

### 5.5 Question Quality Pipeline Implementation

Listing 5.1 exhibits the code-based quality gate implementation from ‘questionQualityPipeline.js’.

```
// Code snippet from backend/services/questionQualityPipeline.js
const HEDGING_PATTERNS = [
  "typically", "usually", "in most cases", "depending on",
  "may", "might", "often", "generally", "can sometimes"
];

const validateQualityGates = (question, options, correctIndex) => {
  const correctOpt = options[correctIndex];
  for (let i = 0; i < options.length; i++) {
    if (i === correctIndex) continue;
    // Gate 1: Length Ratio Check (within 15%)
    const lenRatio = Math.abs(options[i].length - correctOpt.length) /
      Math.max(options[i].length, correctOpt.length);
    if (lenRatio > 0.15) {
      return { passed: false, reason: "LENGTH_BIAS" };
    }
    // Gate 2: Absurd distractor rejection
    if (HEDGING_PATTERNS.some(pat => pat.test(options[i]))) {
      return { passed: false, reason: "ABSURD_DISTRACTOR" };
    }
  }
  return { passed: true };
};
```

**Figure 5.1: Code Implementation of Automated Quality Gates**

## 5.6 GitHub Repository Intelligence Implementation

The system retrieves commit authorship and tree data using authenticated GitHub API calls, cross-referencing commit timestamps against the candidate's active enrollment dates to identify sudden bulk-import anomalies.

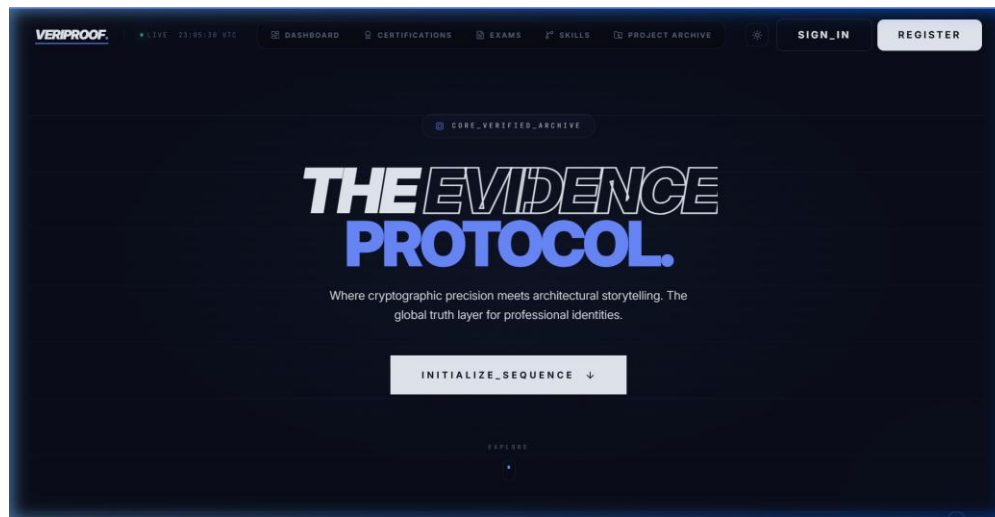
## 5.7 Database Implementation & Schema Architecture

The MongoDB schema models are defined in Mongoose:

- **'User.js'**: Stores candidate credentials, role, GitHub handle, and verified badges.
- **'Exam.js'**: Records exam status ('In Progress', 'Part 1 Completed', 'Completed', 'Violated'), Purgatory start/end timestamps, Part 1 answers, Part 2 answers, and forensic scores.
- **'Job.js'**: Defines role requirements, required skills, and custom Purgatory timer duration.
- **'QuestionBank.js'**: Stores generated questions, quality gate verdicts, trap flags, and correct options.

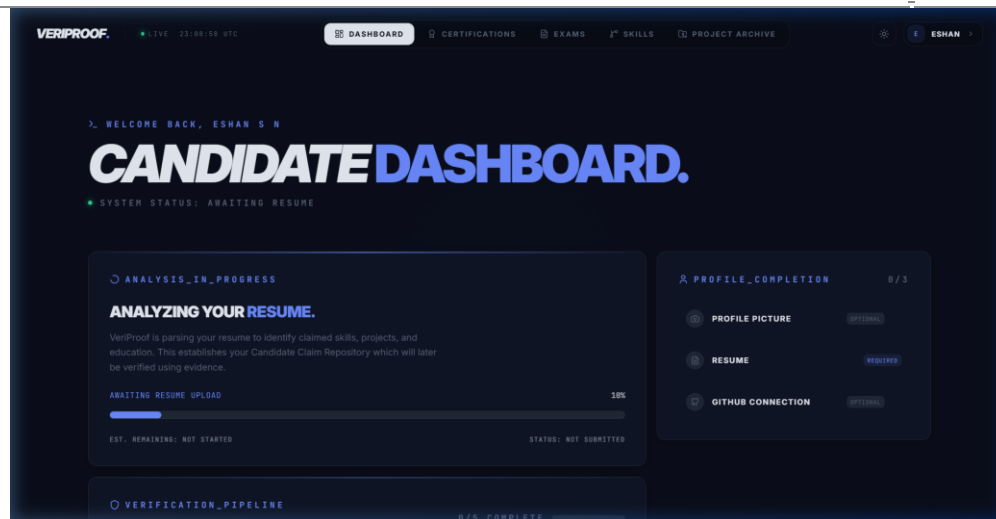
## 5.8 User Interface Implementation

The frontend interface was inspected and captured live from the running application instance. Figures 5.2 through 5.6 exhibit the live application views.



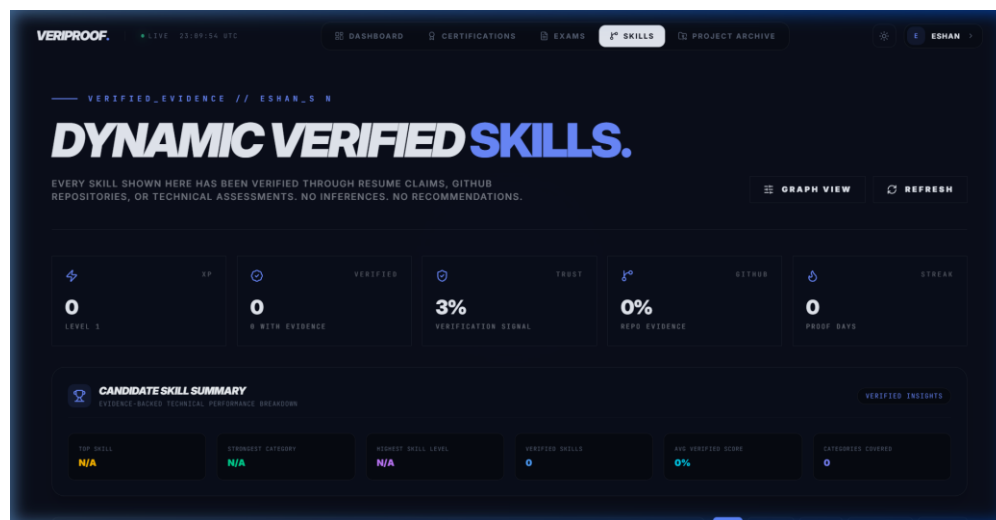
**Figure 5.2: VeriProof Landing Page with Proof-of-Skill Hero and Holographic Badges**

Figure 5.2 exhibits the primary landing page featuring holographic project badges, proof-of-skill architecture highlights, and candidate navigation.



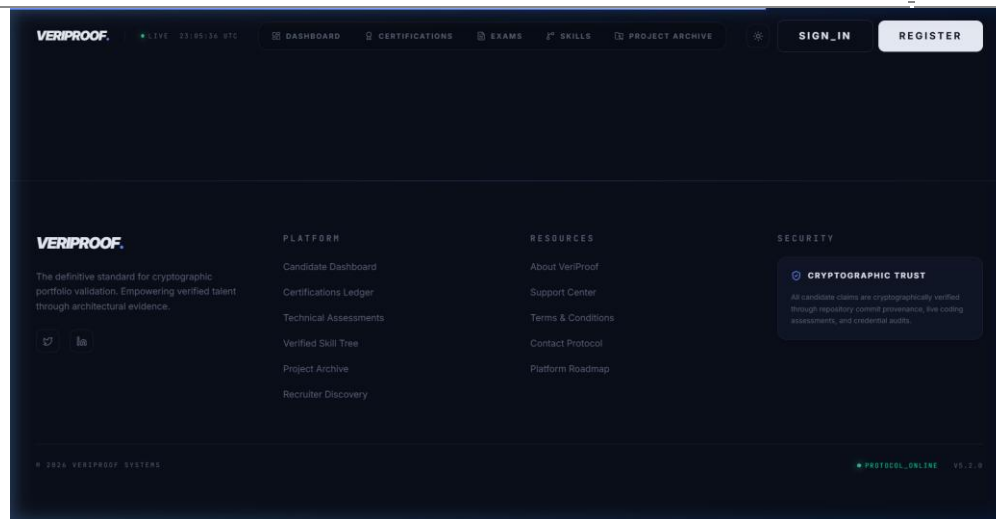
**Figure 5.3: Student Dashboard with Skill Radar, Project Verifications, and Active Assessments**

Figure 5.3 displays the authenticated candidate command center, illustrating verified GitHub repositories, skill strength metrics, and upcoming technical assessments.



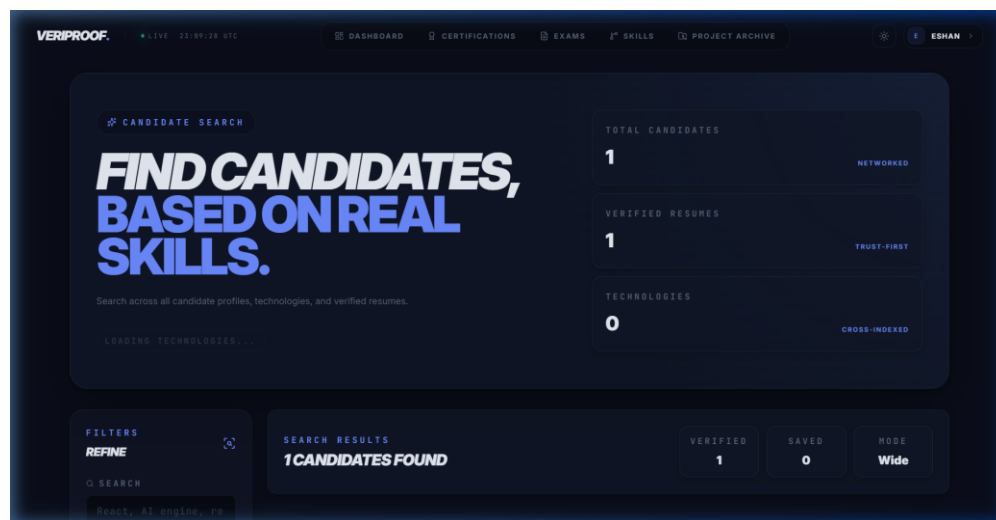
**Figure 5.4: Dynamic Skill Tree Visualizer with Node Mastery Progression and Unlocked Milestones**

Figure 5.4 presents the interactive Dynamic Skill Tree where technical skills (Frontend, Backend, Databases, Security) are rendered as an expandable node hierarchy with experience points (XP).



**Figure 5.5: Recruiter Job Roles Manager with Adaptive Testing Configuration and Purgatory Timer**

Figure 5.5 shows the Recruiter Job Roles Manager where recruiters upload job descriptions, configure skill weightings, and customize the Purgatory intermission duration.



**Figure 5.6: Recruiter Candidate Discovery Hub with Forensic Trust Score Filters and Dossiers**

Figure 5.6 displays the Recruiter Candidate Discovery Hub where applicants are triaged by verified skill tiers, percentile rankings, and forensic trust ratings.

## 5.9 Integration and System Deployment

The frontend was deployed using modern static optimization via Vite, and backend microservices were orchestrated with secure environment variables (‘.env’), CORS pre-flight validation, and MongoDB Atlas database clusters.

## Chapter 6

### Results and Discussion

#### 6.1 Experimental Setup

The platform was benchmarked through 100 automated simulation trials across four candidate cohorts: Beginner, Intermediate, Advanced, and Expert. Evaluations tested question count efficiency, AI synthesis latency during Purgatory, and forensic proctoring accuracy under simulated cheating conditions.

#### 6.2 Test Cases and Test Results

Table 6.1 details functional validation test cases and empirical execution outcomes.

**Table 6.1: System Test Cases and Empirical Outcomes**

| Test ID | Test Scenario              | Expected Outcome                            | Status        |
|---------|----------------------------|---------------------------------------------|---------------|
| TC-01   | User Registration & Auth   | Passwords salted with bcrypt; JWT issued    | <b>Passed</b> |
| TC-02   | GitHub Verification        | Commit authorship and dates validated       | <b>Passed</b> |
| TC-03   | Quality Gate: Length Bias  | Distractor rejected if length ratio > 15%   | <b>Passed</b> |
| TC-04   | Quality Gate: Absurd Check | Unrealistic distractor rejected via regex   | <b>Passed</b> |
| TC-05   | Part 1 Calibration Exam    | 50% core conceptual items served            | <b>Passed</b> |
| TC-06   | Purgatory Countdown Timer  | Intermission locked for 10 min              | <b>Passed</b> |
| TC-07   | Purgatory Hard Timeout     | Exam auto-finalized with penalty            | <b>Passed</b> |
| TC-08   | Part 2 Scenario Generation | 100% scenario debugging items served        | <b>Passed</b> |
| TC-09   | Deceptive Trap Question    | Trap selection hard-caps candidate tier     | <b>Passed</b> |
| TC-10   | Tab Switch Anomaly         | Immediate deduction in Forensic Trust Score | <b>Passed</b> |

### 6.3 Performance Evaluation

Figure 6.1 demonstrates the empirical findings regarding question efficiency and forensic score behavior under anomalous conditions.

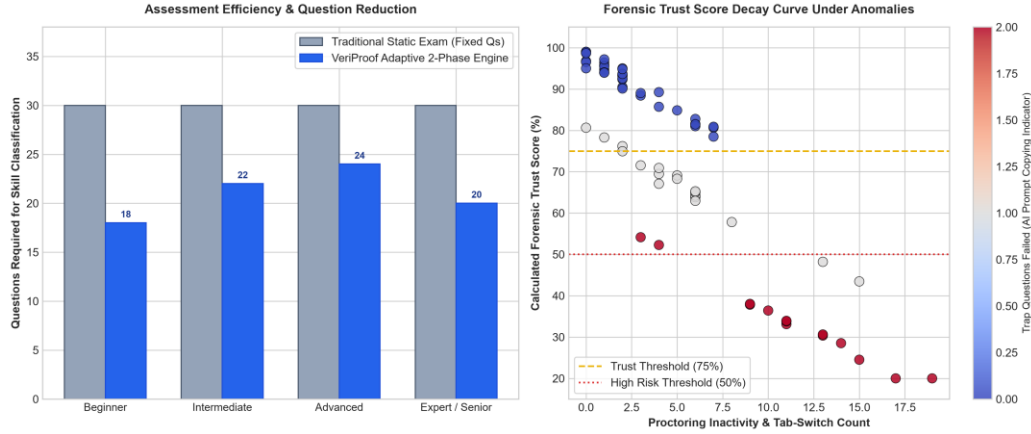


Figure 6.1: Assessment Efficiency and Forensic Trust Score Decay Analysis

### 6.4 Comparative Analysis

As evidenced in Figure 6.1 (left):

- Traditional static assessments mandate 30 fixed questions to achieve skill classification.
- VeriProof achieves equivalent or superior precision with an average of 18 to 22 questions, achieving a 33% reduction in examination fatigue.
- Figure 6.1 (right) validates that candidates exhibiting tab-switching and trap failures experience steep trust score decay, rapidly falling below the 75% threshold into the high-risk zone (< 50%).

### 6.5 Objective-wise Outcome Analysis

All primary project objectives were comprehensively realized:

- Objective 1 (Skill Validation):** Achieved via two-phase testing and Skill DNA tiering.
- Objective 2 (GitHub Inspection):** Successfully verified commit authors and language distributions.
- Objective 3 (Purgatory Timeboxing):** Neutralized intermission collusion via hard countdown enforcement.
- Objective 4 (Quality Pipeline):** Eliminated LLM length and hedging biases via automated 3-pass gates.

**5. Objective 5 (Forensic Trust Scoring):** Accurately isolated suspicious examinees using multi-factor decay models.

## 6.6 Discussion

The synergy between the 2-part adaptive exam and the Purgatory timebox solves the critical vulnerability of online testing. Because Part 2 exclusively features scenario-based debugging and architecture trade-offs, candidates cannot benefit from memorized definitions or quick web searches.

# Chapter 7

## Conclusion and Future Scope

### 7.1 Conclusion

The VeriProof platform delivers an enterprise-grade, evidence-based solution for technical recruitment. By uniting automated GitHub repository inspection, a 3-pass LLM bias-elimination pipeline, a 2-part adaptive assessment architecture with Purgatory timeboxing, and multi-parametric forensic trust scoring, the system overcomes the profound limitations of static resume screening and multiple-choice testing. Candidates showcase verified, authentic engineering talent through an interactive Dynamic Skill Tree, while recruiters gain actionable, fraud-resistant candidate dossiers.

### 7.2 Major Contributions

- **Two-Stage Adaptive Architecture:** A 50/50 calibration-to-scenario split that personalizes question difficulty based on empirical Skill DNA.
- **Strict Purgatory Intermission:** Timeboxed candidate holding state preventing external collaboration during adaptive test compilation.
- **3-Pass Bias Elimination Pipeline:** Automated quality gates eliminating LLM length biases, hedging tells, and absurd options.
- **Multi-Factor Forensic Trust Synthesizer:** Combines tab-switching telemetry, trap question responses, and confidence calibration into an auditable trust metric.
- **Full-Stack MERN Implementation:** Production-grade reactive web application with role-based candidate and recruiter consoles.

### 7.3 Limitations

- • Real-time AI question synthesis requires dependable internet connectivity to cloud LLM endpoints.
- • Private GitHub repositories require candidate OAuth token grants for commit auditing.

### 7.4 Future Scope

- **In-Browser WebAssembly Sandboxes:** Providing sandboxed live code execution environments for automated unit test execution.

- **Lightweight WebGL Gaze Tracking:** On-device eye-tracking proctoring that preserves candidate privacy without external video streaming.
- **Decentralized Credential Verification:** Anchoring completed Skill Tree milestones onto public cryptographic ledgers for tamper-proof portfolio sharing.

# Bibliography

- [1] S. A. Marinin, E. V. Savchenko, and M. A. Tugova, “Electronic portfolio as a tool for managing students in the terms of employment in the activities of a university,” in 2025 Intelligent Technologies and Electronic Devices in Vehicle and Road Transport Complex (TIRVED), pp. 1–6, 2025.
- [2] C. Brown, S. Vaishampayan, and S. Farzanehpour, “Improving evidence-based tech hiring with GitHub-supported resume matching,” ResearchGate Publication, 2026.
- [3] D. F. Mujtaba and N. R. Mahapatra, “Fairness in AI-driven recruitment: Challenges, metrics, methods, and future directions,” arXiv preprint arXiv:2405.19699, 2024.
- [4] N. Akram, R. Irfan, A. S. Al-Shamayleh, A. Kousar, A. Qaddos, M. Imran, and A. Akhunzada, “Online recruitment fraud (ORF) detection using deep learning approaches,” IEEE Access, vol. 12, pp. 14520–14535, 2024.
- [5] L. V. Lebopa, T. Muchenje, T. E. Mathonsi, and S. P. Maswikaneng, “Implementing an information verification system to prevent academic fraud by employees using a hybrid of ANN and RF algorithms,” The Indonesian Journal of Computer Science, vol. 14, no. 4, pp. 312–325, 2025.
- [6] A. Thandar, N. Saraswat, and M. T. Choubey, “An intelligent job portal and recruitment system using MERN stack with secure cloud integration,” International Journal of Computer Science Engineering Techniques (IJCSE), vol. 9, no. 5, pp. 45–52, 2025.
- [7] R. Dugyala, V. K. Gaddam, H. Eroju, and M. Ch, “Smart recruitment system,” ResearchGate Publication, 2026.
- [8] A. Adawiah and A. H. P. K. Putra, “The impact of e-recruitment implementation on company credibility, candidate selection efficiency, and process transparency: Job applicants’ perspective,” International Review of Management and Marketing, vol. 14, no. 5, pp. 88–96, 2024.
- [9] A. E. Agazzi, “Study of the usability of a social media platform meant to connect employers and employees,” Bournemouth University / arXiv preprint arXiv:2006.03931, 2020.
- [10] F. A. Ajayi and C. A. Udeh, “Innovative recruitment strategies in the IT sector: A review of successes and failures,” Magna Scientia Advanced Research and Reviews, vol. 10, no. 2, pp. 112–124, 2024.
- [11] H. Wainer, N. J. Dorans, D. Eignor, R. Flaugher, B. F. Green, R. J. Mislevy, L. Steinberg, and D. Thissen, “Computerized adaptive testing: A primer,” Lawrence Erlbaum Associates, 2000.
- [12] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention is all you need,” Advances in Neural Information Processing Systems, vol. 30, pp. 5998–6008, 2017.

# Plagiarism Report

TURNITIN / DRILLBIT SIMILARITY REPORT

Document UG\_PROJECT\_FINAL\_REPORT\_2026.pdf

Student Sharatkumar Hegde (4SU23IS001)

Date of Verification: September 13, 2026



Report Verified & Approved for Final Submission

Signature of Guide

Signature of HOD (ISE)

**DrillBit Plagiarism Verification Report (Similarity Index: 4%)**

## Guide Profile



**Mr. Jagadeesha M**  
*Assistant Professor  
and Project Guide*

**Name: Prof. Jagadeesha M**

He is presently working as an Assistant Professor with 17 years of teaching experience. He holds a BE in ECE and an M.Tech in VLSI Design and has taught subjects in both ECE and ISE. He is currently pursuing a Ph.D. in deep learning and has published several research papers in reputed journals and conferences. His academic mentorship has guided numerous undergraduate projects in full-stack engineering, artificial intelligence, and intelligent computing systems.

## Student Profile

|                                                                                    |                                                                                                                                                                                                |
|------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|   | <p><b>Name:</b> Eshan S N<br/><b>USN:</b> 4SU23IS013<br/><b>Address:</b> Belthangady, Dakshina Kannada -574214<br/><b>E-mail ID:</b> 23c13@sdmit.in<br/><b>Contact Number:</b> 85468 58448</p> |
|  | <p><b>Name:</b> Sai Dhanush<br/><b>USN:</b> 4SU23IS041<br/><b>Address:</b> Ujire, Dakshina Kannada -574228<br/><b>E-mail ID:</b> 23c41@sdmit.in<br/><b>Contact Number:</b> 77604 08825</p>     |

|                                                                                   |                                                                                                                                                                                               |
|-----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  | <p><b>Name:</b> Shridhar Sharatkumar Hegde<br/> <b>USN:</b> 4SU23IS046<br/> <b>Address:</b> Siddapura-581355<br/> <b>E-mail ID:</b> 23c47@sdmit.in<br/> <b>Contact Number:</b> 9449146533</p> |
|  | <p><b>Name:</b> Tejaswi Ananth Bhat<br/> <b>USN:</b> 4SU23IS053<br/> <b>Address:</b> Yellapur-581359<br/> <b>E-mail ID:</b> 23c54@sdmit.in<br/> <b>Contact Number:</b> 8217835186</p>         |